

WEEK 12

全链路可观测性

故障定位的显微镜——给 Agent 系统装上“X 光”，让排障从“小时级”降到“分钟级”

01 OTel

协议基础

OpenTelemetry + OpenInference

02 Spans

Span 设计

6 段必备 · 读起来像故事

03 Dash

仪表盘

5 个必看 · DoW 对比

04 Alert

告警 SLO

SLO + 错误预算 · 4 级

05 Replay

Bad Case 复盘

5 步流程 · 反例库复利

本周划重点

Week 11 让你知道系统“好不好”。这一周解决另一个要命的问题——它出事了，你能不能在客户骂完之前找到“卡在哪、为什么”？一个请求穿越 10 多个服务，光看日志就是黑灯瞎火。我们给它装上一台显微镜，让任何一条请求都能完整还原。

LESSON 01 · OTEL + OPENINFERENCE

可观测性的"水电煤": OpenTelemetry + OpenInference 协议基础

Week 11 给质量装了刻度, 这一周给系统装上"X 光"——先把协议地基打好

真实场景 · 上线后最怕听到的那句反馈

"今天 Copilot 答得很慢。" "昨天那条回答好像有问题。" ——你品品这两句, 一个细节都没有, 可你得在 30 分钟内回答客户: "是检索慢、重排慢、生成慢, 还是工具卡了?" 我见过太多团队这时候是怎么干的: 几个工程师围在一起翻日志, 一个请求穿越十几个服务, 日志散在七八个系统里, 翻到天黑也对不上号。没有可观测性, 你就是在黑灯瞎火里摸电闸。这一讲讲清楚: 怎么用 OpenTelemetry + OpenInference, 给 Agent 系统装上一台能看穿每一条请求的 X 光机。

咱们这节聊 4 件事:

- Logs / Metrics / Traces 三类信号各解决什么——为什么 Agent 时代 Traces 是必需
- OTel 核心模型: Trace / Span / Context, 3 个 API 就能把全链路追踪上车
- OpenInference 怎么把 OTel 扩展成 LLM 专用协议——以及它和 OTel 官方 GenAI 约定的关系
- 在 OmniSupport 里一行代码自动 instrument, 业务代码完全不动

LOGS ARE NOT ENOUGH

LLM/Agent 时代"打日志"不够用了——必须升级到 trace + span + context

传统 Web 一个请求穿 2-3 个服务，日志够用；Agent 一个请求穿 8-15 个 hop

一条 Agent 请求的真实路径，光靠日志你根本串不起来：



关键判断

8-15 个 hop，每个都有耗时、错误、依赖。光看日志，你既找不到"慢在哪一段"，也没法把"用户问题 ↔ LLM 决策 ↔ 工具调用"关联起来——它们躺在不同服务的不同日志文件里，时间戳还对不齐。所以必须升级到分布式追踪：一个 trace_id 贯穿全链，每一段一个 span 记录细节。这就是 OTel 设计的初衷，OpenInference 在它上面加了一层 LLM 专用语义。记死这句——Agent 时代，trace 比 log 重要 10 倍，因为 log 是"点"，trace 是"因果链"。

THREE PILLARS

可观测性的 3 个支柱——Logs / Metrics / Traces，各管一摊，缺一不可

别只押一样：日志看点、指标看面、追踪看因果链

Logs 日志

- 事件级文本
- "12:05 调了 RAG 服务"
- 管：单点事件回顾
- 短板：无法跨服务关联
- 工具：ELK / Loki

Metrics 指标

- 数值时序
- "P99 2.3s / qps 120"
- 管：趋势 + 容量
- 短板：知道出事、不知在哪
- 工具：Prometheus

Traces 追踪

- 请求级因果链
- "这次穿 8 个服务、慢在 rerank"
- 管：跨服务故障定位
- 强项：Agent 时代的命门
- 工具：OTel + Jaeger/Tempo

关键判断

Metrics 告诉你"系统发烧了"，但烧在哪个器官，只有 Traces 能指出来。Agent 时代的排障，是从一条 trace 的时间轴上"看"出根因，不是在日志里"搜"。三样一起用、用同一个 trace_id 串起来，才是完整的可观测。

OTEL CORE MODEL

OTel 核心模型 Trace / Span / Context——10 行代码讲清楚

理解这 3 个概念，所有可观测代码长得都一样

```
from opentelemetry import trace
tracer = trace.get_tracer(__name__)

# Trace = 一次完整请求 = 一棵 span 树
# Span = 请求里的一个具体步骤
# Context = 跨服务传递的 trace_id 信封

with tracer.start_as_current_span("rag.query") as span:
    span.set_attribute("user.query", query)

    with tracer.start_as_current_span("retrieve.hybrid") as s:
        s.set_attribute("retrieval.top_k", 50)
        candidates = hybrid_search(query)
        s.set_attribute("retrieval.hits", len(candidates))

    with tracer.start_as_current_span("rerank.cross") as s:
        s.set_attribute("rerank.model", "bge-reranker-v2-m3")
        top5 = reranker(query, candidates)

    with tracer.start_as_current_span("llm.generate") as s:
        s.set_attribute("llm.model", "gpt-5")
        s.set_attribute("llm.input_tokens", count(prompt))
        answer = llm.complete(prompt)

span.set_status(trace.Status(trace.StatusCode.OK))
```

老司机解读

start_as_current_span 自动建父子关系 · rag.query 是父，
retrieve/rerank/llm 是子，自动嵌成一棵树

set_attribute 加结构化属性 · 之后在 Phoenix / Jaeger 里直接按字段查询过滤

一个 trace_id 自动在所有 span 间传递 · 这就是 OTel 的 context propagation——你啥都不用做

span 自动记开始/结束时间 · 时间轴上一眼看出“哪段慢”

简洁到极致 · tracer / span / attribute 三个 API，就把 RAG 全链路追踪上了

OPENINFERENCE

OpenInference：把 OTEL 扩展成 LLM 专用——LLM 字段标准化，跨工具互通

OTel 的 span 字段是"业务无关"的；LLM 那些特殊数据，得有人来定标准

字段类别	标准属性	为什么需要统一
LLM Call	llm.model_name / invocation_parameters	跨工具识别用的什么模型
Tokens	llm.token_count.prompt / completion / total	统一算成本
Messages	llm.input_messages / output_messages	跨工具看 prompt
Tools	tool.name / description / json_schema	跨平台识别工具调用
Retrieval	retrieval.documents / score	RAG span 标准字段
Embeddings	embedding.model / text	Embedding 调用标准

老司机说

prompt / completion / tokens / temperature 这些，OTel 标准里没规定——各家自己造字段，结果工具间不互通、换个平台数据全废。2024 年 Arize 主导推出 OpenInference 把这些钉成标准，Phoenix / LangSmith / Langfuse 都原生支持——你 emit 一次，多个平台都能消费。这就是"协议层统一"的价值。

REAL ARTIFACT

OmniSupport 里的 OTel + OpenInference 接入——一行代码自动追踪

业务代码一个字不改, Instrumentor 自动注入符合标准的 span

```
# pipelines/observability/setup.py
from opentelemetry import trace
from opentelemetry.sdk.trace import TracerProvider
from opentelemetry.sdk.trace.export import BatchSpanProcessor
from opentelemetry.exporter.otlp.proto.grpc.trace_exporter \
    import OTLPSpanExporter
from openinference.instrumentation.openai import OpenAIInstrumentor
from openinference.instrumentation.langchain import LangChainInstrumentor

def setup_observability():
    provider = TracerProvider()
    exporter = OTLPSpanExporter(
        endpoint="https://phoenix.internal:4317")
    provider.add_span_processor(
        BatchSpanProcessor(exporter)) # 异步批量, 不阻塞业务
    trace.set_tracer_provider(provider)
    # 一行 instrument —— 自动追踪所有 LLM 调用
    OpenAIInstrumentor().instrument()
    LangChainInstrumentor().instrument()
    return trace.get_tracer("omnisupport.copilot")

# 应用入口调一次; 业务代码完全不动:
response = openai_client.chat.completions.create(
    model="gpt-5", messages=[...])
# 自动生成 span: llm.model_name / token_count / messages
```

老司机解读

OpenAIInstrumentor().instrument() · 一行代码自动追踪所有 OpenAI/Anthropic 调用, 业务代码零改动

OpenInference 字段自动填好 · llm.model_name / token_count.* 不用手写 set_attribute

BatchSpanProcessor 异步批量发送 · 不阻塞业务、不增加延迟

一份设置多个平台消费 · Phoenix / Jaeger / Langfuse 走同一份 OTLP 协议, 换平台不改代码

这一刻起你的系统有了"X 光" · 任何一条请求都能完整还原

INDUSTRY 2026

OTel + OpenInference 已是事实标准——自研可观测协议在 LLM 时代是工程债

2026 一个关键动向：OTel 官方 GenAI 语义约定正在和 OpenInference 收敛

标准 / 平台	能力	工程含义
OpenTelemetry (CNCF 毕业)	继 Prometheus 之后第 2 个毕业的可观测项目	所有云厂商都支持，事实标准
OpenInference (Arize 2024)	LLM 字段标准化，Phoenix/LangSmith/Langfuse 原生支持	现在就能用、生态最成熟
OTel GenAI 语义约定 (gen_ai.*)	官方在推 invoke_agent / execute_tool / chat span	2026 仍 experimental→stable，与 OpenInference 收敛中
Phoenix / Langfuse / Braintrust / Datadog	后端兼容 OTel，前端做 Agent 专用可视化	一次埋点，跨工具自由切换

记死

2026 年的事实 → 直接 OTel + OpenInference 起步，别自研协议。一个要跟的趋势：OTel 官方的 GenAI 语义约定正在把 LLM/Agent 的 span 字段收进"国标"，目前还在 experimental→stable 过渡，和 OpenInference 在收敛。我的建议：当下用 OpenInference（生态成熟），同时关注 gen_ai.* 的进展——埋点抽象一层，将来切换无痛。

LESSON 02 · SPAN-LEVEL TRACING

Span 级追踪设计：让每一段推理都能"被时间轴看清"

上一讲把 OTel 接进来了——接进来不等于用对

真实场景 · 8 个一模一样的方框

我 review 过一个团队的 trace，OTel 接得挺好，可一打开 Phoenix 时间轴——8 个 span 全叫 "process"、"call"、"run"，属性里就俩字段 input/output，各塞了几千字符。点开半天才搞清楚哪个是召回、哪个是生成。这叫"伪可观测"：看着有 trace，实际等于没有。span 设计得好不好，决定了你排障是 5 分钟还是 5 小时。这一讲讲清楚：怎么为 RAG / Tool / HITL 设计"有信息量"的 span——点开时间轴，一眼看出"问题在哪、为什么、影响什么"。

咱们这节聊 4 件事：

- span 设计的 3 大原则：命名 / 属性 / 状态——为什么"读起来像故事"是第一原则
- RAG 全链路 6 段 span（query→intent→retrieve→rerank→generate→audit）是故障定位最小集
- Tool / HITL 的 span 怎么配合 Week 10 的工具契约和 HITL——别留黑盒
- span 属性的"预算"：既要全又要快——preview + len + artifact_url 的标准模式

READ LIKE A STORY

好的 span = "读起来像故事"——名字 + 关键属性，一眼说清这一步干了什么

坏 span 让你点开看半天，好 span 让你扫一眼就懂

坏 span

- name = "process"
- attr = {input, output}
- 各塞几千字符
- 点开看半天不知干嘛

好 span

- name = "rag.retrieve.hybrid"
- strategy=vec+bm25+rrf
- vec_hits=47 / bm25=39 / fused=53
- filter=vip · elapsed=187ms

一眼读出

- 用了什么策略
- 各路返回多少
- 过滤了什么
- 花了多久 → 5 分钟定位

关键判断

span 命名我有个土办法：照"好函数名"的标准来——layer.action.strategy (rag.retrieve.hybrid)，而不是 process / call / run。属性照"好日志字段"的标准来——存关键参数、关键结果、关键决策，别存原文。一句话：让运维半夜被叫醒、迷迷糊糊扫一眼时间轴就能读懂的 span，才是好 span。

DESIGN PRINCIPLES

Span 设计的 3 大原则——命名 / 属性 / 状态，缺一不可

每一条都有现成的类比，照着抄就行

Naming 命名

- 用"层.动作.策略"
- 如 rag.retrieve.hybrid
- 不写 process / call / run
- 可按前缀过滤
- 类比：好函数命名

Attributes 属性

- 关键参数/结果/决策
- 不存原文（爆字段）
- 不存 PII（合规）
- 加 _count / _ms 后缀
- 类比：好日志字段

Status 状态

- OK / ERROR + error_type
- 业务级状态码
- 如 insufficient_evidence
- Phoenix 可按状态过滤
- 类比：HTTP status

关键判断

状态码这条最被低估：别只有 OK/ERROR，要带业务级 error_type——"insufficient_evidence""tool_timeout""low_confidence"。出错时一眼知道"为什么错"，而不是只知道"错了"。这一个字段，能省你半小时翻日志。

RAG SPAN CHEATSHEET

RAG 全链路 6 段标准 span——经验证的"故障定位最小集"，少一段就有盲区

6 段合起来能回答：慢在哪？召回准不准？重排丢多少？生成花多少 token？审计写了没？

Span Name	父子	关键属性	定位什么
rag.query	根	user_id / query / tenant / role	整体请求入口
rag.intent_route	子	intent / confidence / route	走 RAG 还是 Tool
rag.retrieve.hybrid	子	top_k / vec_hits / bm25_hits / fused	召回阶段
rag.rerank.cross	子	model / kept / threshold / dropped	精排阶段
rag.generate.llm	子	model / in_tokens / out_tokens / temp	生成阶段 + 成本
rag.audit.write	子	trace_id / release_id / evidence_count	审计完整性（接 Week 6 OpenLineage 血缘）

老司机说

这张"速查表"我建议直接贴墙上。为什么是这 6 段不是 5 段也不是 10 段？因为它正好对齐 Week 8 那条"检索+重排+生成"流水线 + Week 10 的审计——每段对应一个你会真去查的问题。多了是过度埋点（贵、慢），少了就有盲区（出事查不到）。

RAG SPANS REAL CODE

OmniSupport 里的 RAG 完整 span 实现——三路召回数字一目了然

LLM 那段不用手写 span, Instrumentor 自动注入

```
# services/rag/traced.py
tracer = trace.get_tracer("omnisupport.rag")

def rag_query(query, tenant, role):
    with tracer.start_as_current_span("rag.query") as root:
        root.set_attribute("user.tenant", tenant)
        root.set_attribute("query.text", query[:200]) # 只存前200字
        root.set_attribute("query.len", len(query))

        with tracer.start_as_current_span("rag.retrieve.hybrid") as s:
            s.set_attribute("retrieval.strategy", "vec+bm25+rrf")
            vec = pgvector.search(query, top_k=50)
            bm25 = es.bm25(query, top_k=50)
            fused = rrf_fusion([vec, bm25])
            s.set_attribute("retrieval.vec_hits", len(vec))
            s.set_attribute("retrieval.bm25_hits", len(bm25))
            s.set_attribute("retrieval.fused_total", len(fused))

        with tracer.start_as_current_span("rag.rerank.cross") as s:
            s.set_attribute("rerank.model", "bge-reranker-v2-m3") # 或 Cohere Rerank
            top5 = reranker.rerank(query, fused, top_k=5)
            s.set_attribute("rerank.kept", len(top5))
            s.set_attribute("rerank.dropped", len(fused)-len(top5))

        answer = openai_client.chat.completions.create(...) # 自动 span
        root.set_status(trace.Status(trace.StatusCode.OK))
    return answer
```

老司机解读

root span "rag.query" 是整棵树入口 · 下游 span 自动归到这棵树下

query.text 只存前 200 字 + query.len · 既能 review 又不爆字段

retrieval 三个字段 (vec/bm25/fused) · 在 Phoenix 上立刻看出"哪一路召回出了问题"

rerank.kept / dropped · 阈值合不合理一目了然——丢太多就是阈值太狠

LLM 调用不用手写 span · OpenAIInstrumentor 自动生成符合 OpenInference 标准的 span

TOOL & HITL SPANS

Tool / HITL 的 span——Agent 时代的关键扩展，配合 Week 10 工具契约

工具慢/失败/路由错、HITL 卡死——没 span 全是黑盒

Span Name	关键属性	回答什么问题
tool.call.{name}	tool.name / args_hash / result_code / elapsed	工具调用快不快 / 对不对
tool.fallback	level / primary_error / final_level	降级链走到第几级
tool.idempotent_check	idem_key / hit / args_hash	命中幂等缓存了吗
hitl.request	risk_category / approval_layer / sla_min	HITL 触发原因
hitl.wait	notified_at / waited_ms	审批人响应快不快
hitl.decided	approver / decision / reason	审批结果 + 责任人

老司机说

这 6 类 span 把"Agent 决策 + 工具执行 + 人工介入"完整记下来——不管是本地工具还是 MCP server 暴露的工具，都走同一套 span。我管它叫受控 Agent 的"行车记录仪"。出了事故，hitl.wait 的 waited_ms 一看就知道是不是审批卡了 40 分钟（Week 10 那个真实事故，LangGraph interrupt 的等待就发生在这段），tool.fallback 的 final_level 一看就知道降到了第几级。和 Week 10 的工具契约 audit_fields 完全对应。

ATTRIBUTE BUDGET

Span 属性的"预算"——既要全又要快，别把 trace 写成数据库

一个 span 建议 < 20 个属性 + 总字节 < 2KB，否则上报存储成本爆炸

```
# 错误：属性太多 → 上传慢 + 存储贵 + 难读
span.set_attribute("full_prompt", prompt)    # 几千字符
span.set_attribute("full_response", response) # 几千字符
span.set_attribute("all_candidates", json.dumps(candidates))

# 正确：选择性 + 上限
span.set_attribute("prompt_preview", prompt[:200])
span.set_attribute("prompt_len", len(prompt))
span.set_attribute("response_preview", response[:200])
span.set_attribute("response_len", len(response))
# 候选只存数量 + 前 3 个 ID，不存完整内容
span.set_attribute("candidate_count", len(candidates))
span.set_attribute("top_3_chunk_ids",
    [c.chunk_id for c in candidates[:3]])

# 完整内容 → "link to artifact"
# 把完整 prompt/response/candidates 存到对象存储 S3
artifact_url = upload_to_s3(prompt, response, candidates)
span.set_attribute("artifact.url", artifact_url)
# Phoenix UI 点 artifact.url 跳到详情；trace 表本身保持小巧高性能
```

老司机解读

_preview + _len 组合 · 既能 review 又不爆字段——这是最高频的实战技巧

top_3_chunk_ids 只存 ID 不存原文 · 复盘时按 ID 再去拉原文

artifact_url 指向对象存储 · 这就是 "hot trace + cold storage" 标准模式：
trace 表只放热数据，大块内容沉到 S3

Phoenix / LangSmith / Arize 都支持 link-to-artifact · 点一下跳详情

我踩过的坑 · 早期把整段 prompt 塞 span，trace 后端存储一个月涨了 5 倍；换成 preview+artifact 后存储省了约 80%、查询快一倍

TRACE TIMELINE

一条完整 trace 在 Phoenix 时间轴上长什么样——这就是"故障定位的显微镜"

任何一段慢、错、漏，点开都能看到具体属性



关键判断

你看这条时间轴——一眼就知道 2.4 秒里，1850 毫秒花在了 llm.generate 上。所以要优化延迟，别瞎调召回，先动生成（换模型 / 减 max_tokens / 流式输出）。这就是"看时间轴"和"猜"的区别：数据告诉你瓶颈在哪，你才不会把一周时间花在错的地方。Phoenix 上每一段都可点击，点进去就是上一页那些属性。

INDUSTRY 2026

Span 设计的行业最佳实践——照 OpenInference spec 抄，别自己定字段

自定义 span 字段是工程债，跨工具就废

来源	能力	工程含义
OpenInference spec	RAG / LLM / Tool 各类 span 的标准命名	跨工具通用，强烈建议直接遵守
Anthropic / OpenAI 原生 trace	响应里自带 reasoning span + tool_use span	开箱即用，不用自己 wrap
Phoenix (Arize)	OpenInference span 专用 LLM 可视化	prompt/response 一键展开，token 成本立现
OTel GenAI semconv	gen_ai.* 统一 span/attribute 命名	2026 仍 experimental，但方向是它

记死

2026 年的共识 → span 字段直接照 OpenInference spec 抄，将来无缝迁到 OTel GenAI 官方约定。别犯那个最常见的错：自己拍脑袋定一套字段名，用着用着发现换个可观测平台全部对不上、历史数据全废。协议这种东西，跟标准走永远比自创省事。

PITFALLS

Span 设计的 5 个常见反模式——"存原文 + 有 PII"最危险

既贵又违规，span 设计第一原则：精简 + 脱敏

反模式	具体表现	后果	正确做法
名字通用	name = "process" / "call"	时间轴看不出在干嘛	"层.动作.策略"命名
存原文	attr = {full_prompt: "..."} 把 customer.phone 存属性	上传慢 + 存储贵	_preview + _len + artifact_url
有 PII		合规违规（监管事件）	redact 脱敏后再 set
少关键状态	只有 OK / ERROR	错了不知道为什么	加业务级 error_type
不分层	所有 span 平级	看不出依赖关系	父子嵌套，按层组织

踩坑提醒

"有 PII"这条是真出过事的——我见过团队图省事，把整个用户 message 塞进 span 属性，里面有身份证号、手机号，结果 trace 数据进了第三方可观测 SaaS，直接变成一次数据出境 + PII 泄露的合规事件。span 是给"系统"看的，不是存"用户隐私"的。set_attribute 之前先过一遍 redact，这条没有商量。



LESSON 03 · OBSERVABILITY DASHBOARDS

实时监控仪表盘：把百万条 trace 浓缩成 5 个"必看面板"

trace 全上车之后，下一个问题是——这么多数据，到底看哪几张图

真实场景 · 凌晨 3 点，一墙的图看不出问题

我见过一个"很专业"的 dashboard——20 多个面板挤一屏，各种曲线密密麻麻，做的人特别自豪。可真出事那晚，oncall 凌晨 3 点被叫起来，盯着那一墙图，愣是看不出哪里不对。信息全在那，但"看不见"。dashboard 的核心矛盾就在这——"看得多"和"看得清"是反的。生产级 dashboard 不是"展示所有数据"，是"为特定角色解决特定决策问题"。这一讲讲清楚：怎么浓缩出 oncall 30 秒就能判断"系统健不健康"的 5 张图。

咱们这节聊 4 件事：

- 为什么"全部指标摆一墙"反而让人看不见问题——dashboard 是为决策设计的
- oncall / 工程师 / PM 三类角色的面板需求完全不同，必须分开做
- 5 个必备面板：Overview / Quality / Performance / Cost / Errors（SRE 黄金信号 + LLM 质量）
- 为什么 Day-over-Day 对比比"实时数字"更能发现问题



MORE VS CLEARER

Dashboard 的核心矛盾——"看得多" vs "看得清", 必须为不同角色定制

把所有数据堆一墙看起来很专业, 但 oncall 凌晨 3 点根本看不出问题

Oncall

- 看 5 张图
- 判断"系统健不健康"
- 红绿灯 + DoW 对比
- 决策: 要不要升级

工程师

- 看具体 trace + span
- P50/P95/P99 拆解
- 错误类型 + 堆栈
- 决策: 怎么修

PM / 业务

- 看业务指标趋势
- 一次解决率 / CSAT / 成本
- 与上一版对比
- 决策: 要不要砸资源

关键判断

dashboard 设计第一原则: 不同角色、不同面板。同一份 trace 数据, oncall 要的是"红绿灯" (5 张图判断升不升级), 工程师要的是"显微镜" (20+ 详细面板查怎么修), PM 要的是"趋势线" (业务和成本)。把这三拨人的需求塞进同一个 dashboard, 结果就是三拨人都嫌它难用。先问"谁在什么场景下用", 再画图。

FIVE MUST-HAVE PANELS

5 个必备面板——Google SRE 黄金信号 + LLM 质量维度

不管什么 Agent 项目，这 5 张是"系统体检报告"，缺一张有盲区

面板	看什么	关键指标	诊断什么
Overview 总览	系统健不健康	QPS / Error Rate / P99	要不要升级响应
Quality 质量	答得好不好	Faithfulness / Citation / Refuse Rate	要不要回滚
Performance 性能	快不快	各 span 耗时分布	哪一段慢
Cost 成本	贵不贵	Token × 单价（\$2-15/1M tok 量级）	要不要降级模型
Errors 错误	错在哪	错误类型分布 + 趋势	哪类故障在升

老司机说

前 4 张（总览/性能/成本/错误）是 Google SRE "黄金信号"的变体，第 5 张 Quality 是 LLM 时代专属——传统系统不用测"答得对不对"，Agent 系统必须。这 5 张要放在同一屏，切来切去会错过关联：比如成本突然涨、同时 P99 也涨，大概率是召回 top_k 被人调大了——分开看就发现不了这种关联。

PANEL QUERIES

OmniSupport 里 5 个面板的查询逻辑——全部基于 OTEL span 属性，不用单独 ETL

你 Lesson 1/2 埋的那些 span 属性，这里直接出图

```
# observability/dashboards/ —— 全部查 span 属性，无需 ETL
# Panel 1: Overview
"QPS": "count(span='rag.query') / 60s"
"Error Rate": "count(status='ERROR') / count(*)"
"P50/P99": "percentile(elapsed_ms, .5/.99) where span='rag.query'"

# Panel 2: Quality
"Citation Coverage": "count(evidence_count>0) / count(*)"
"Refuse Rate": "count(insufficient_evidence=true) / count(*)"
"Live Faithfulness": "avg(faithfulness) where sampled=true" # 1% 抽样
"Bad Case Rate": "count(user_feedback='thumbs_down') / count(*)"

# Panel 3: Performance —— 各 span 耗时拆解
"retrieve_p99": "percentile(elapsed,.99) where span='rag.retrieve.hybrid'"
"rerank_p99": "percentile(elapsed,.99) where span='rag.rerank.cross'"
"llm_p99": "percentile(elapsed,.99) where span='llm.generate'"

# Panel 4: Cost Panel 5: Errors
"Cost/Query": "sum(prompt_tok*price + completion_tok*price)/QPS"
"Tool Failures": "count(*) by tool_name where status='ERROR'"
"HITL SLA Miss": "count(*) where span='hitl.wait' and waited_ms>900000"
```

老司机解读

所有查询都基于 OTEL span 属性 · 不需要单独 ETL 一份数据——这是统一协议的红利

P99 + P50 双指标 · P99 看“最差用户体验”，P50 看“中位体验”，必须都看

Live Faithfulness 用 1% 抽样 · 在线全量跑 RAGAS/DeepEval 的 LLM-judge 太贵，抽样够用（Week 11 学过）

Tool Failures 按 tool_name 分组 · 立刻看出哪个工具最不稳

waited_ms > 900000 · 就是 Week 10 那个 15 分钟 HITL 超时——埋点串起来了

DAY-OVER-DAY

Day-over-Day 对比——比"实时数字"更能发现问题

"现在 P99=2.1s" 几乎没信息量；"比上周同时段差 30%" 才是问题

对比方式	怎么算	适合	发现什么
同比 DoW	今天 vs 上周同一天同时段	日常运营	周节律突变
环比 HoH	当前小时 vs 上一小时	快速问题发现	突发故障
基线对比	7 日 / 30 日均线	长期趋势	慢慢退化
Release 对比	当前版本 vs 上一 release	发布回归	上线后退化
百分位带	P10-P90 历史带 + 当前点	判断正常波动	识别真异常

老司机说

绝对值会骗人，变化率不会。我的 oncall 面板里每个核心指标都带一条"上周同时段"的虚线——电商有明显周节律，周一上午的 P99 本来就比周日高，你拿固定阈值告警全是误报；拿 DoW 一比，"比上周同时段高 30%"才是真异常。oncall 仪表盘至少要有 3 种对比，少了就只能"靠记忆判断今天正不正常"。



THREE-LEVEL DRILLDOWN

仪表盘的"三层钻取"——总览 → 拆解 → 详情 → 决策

让 oncall 30 秒内从"总览异常"定位到"具体 span", 不再大海捞针



关键判断

这条钻取链是 dashboard 设计的灵魂——每一层都能"点进下一层"。总览看到 Error Rate 飘红 → 点进去看是哪类错 → 点开一条 sample trace 看时间轴 → 点开出错的 span 看属性 → 决定回滚还是热修。最关键的一条：面板必须能"跳转到 trace"。我见过太多 dashboard 看到异常却点不进去，oncall 还得另开一个工具重新搜——那条钻取链一断，30 秒定位就变成 30 分钟。Phoenix / LangSmith 都原生支持双向跳转。

INDUSTRY 2026

LLM 可观测平台已"选型分化"—开源 / 一站式 SaaS / 企业级 APM 三流派

2026 别再自己造可视化，三条路按团队规模选

工具	定位	适用 / 局限
Phoenix (Arize) / TruLens	LLM 可观测开源事实标准、自托管 + feedback function	中型团队 / 自主可控 (UI 朴素)
Langfuse	开源 + LLM 专用 + 自托管友好	要数据驻留 / 合规自托管首选
LangSmith	trace + eval + prompt 一站式 SaaS	LangChain 生态 / 快速验证
Braintrust / Datadog LLM	CI 门禁 + PR 评分 / 企业级 APM 一体化	要发布门禁 / 大企业统一可观测

记死

2026 年的推荐路径 → 自托管/合规优先选 Langfuse 或 Phoenix；LangChain 栈用 LangSmith；要把可观测和"发布门禁/PR 评分"打通，看 Braintrust。它们后端都兼容 OTel，前端做 Agent 专用可视化——你 Lesson 1 埋的那一份 OpenInference span，喂给哪个平台都能用。不要再花三个月自己造一个 dashboard 框架。

PITFALLS

Dashboard 设计的 5 个常见反模式——"全部一墙 + 不分角色"最致命

这是新团队 dashboard 失败的 90% 原因

反模式	具体表现	后果	正确做法
全部一墙	20+ 面板挤一屏	oncall 看不见问题	5 个总览 + 钻取
只看实时	不做 DoW 对比	不知道是否退化	Day-over-Day 必备
只看 P99	忽略 P50 / 错误率	中位体验掉了不知道	P50 + P99 + Error 都看
不分角色	oncall 看工程师面板	关键信号被淹没	分角色定制
面板点不进 trace	看到异常但点不进去	调试要重开工具	面板双向跳转 trace

踩坑提醒

"面板点不进 trace"这条最容易被忽略，却最毁效率。我的硬要求：任何一个总览面板上的异常点，必须能一键跳到对应的 trace 列表，再点进单条 trace 时间轴。这条钻取链断在哪里，oncall 的排障时间就在哪里翻十倍。选工具时这是我第一个验收项——不支持双向跳转的，再便宜也不用。

LESSON 04 · ALERTS & SLO

告警与 SLO：让系统"该叫人时叫人，不该叫人时闭嘴"

大多数团队花 80% 时间建 dashboard、10% 设告警、0% 想 SLO——顺序全反了

真实场景 · 凌晨 3 点 50 条告警，全是误报

我见过最惨的一次 oncall：凌晨 3 点，手机连续炸了 50 条告警，爬起来一看——全是误报，高峰期 P99 自然高一点就触发了阈值。他烦到直接把告警静音了。结果第二天起来，真正的故障——一个工具大面积超时——已经悄悄发了 5 个小时，客户投诉一片。这就是"告警疲劳"：告警太多太吵，人就麻木，真出事反而被淹没。SLO 这套上来，能把误报砍掉 90% 以上。问题的根不在"告警写得不够多"，在于大多数团队设告警靠"经验拍数字"。这一讲讲清楚：为什么 SLO + 错误预算，才是告警真正的科学基础。

咱们这节聊 4 件事：

- 为什么"P99 > 2s 就告警"这种阈值法在 LLM 系统里几乎一定误报
- SLO 三要素：SLI（指标）+ SLO（目标）+ 错误预算——告警的科学基础
- Copilot 的 5 类核心 SLO + 错误预算燃烧速度（burn rate）告警
- 4 级告警（P0/P1/P2/P3）+ 不同路由——避免告警疲劳

SLO, NOT THRESHOLD

告警的真正基础不是阈值——是 SLO + 错误预算 + 燃烧速度

"P99 > 2s 就告警"是 2018 年的做法，在 LLM 系统里 = 疯狂误报

Google SRE 给出的正确答案，三个概念串起来：

SLO 目标

- 业务可接受的范围
- 如"99% 请求 P99 < 3s"
- 从业务诉求来
- 不是越高越好

错误预算

- 100% - SLO
- 如 1% 可以失败
- 是你能"花"的额度
- ——这才是关键

燃烧速度

- 错误预算被消耗多快
- 正常速度 = 不告警
- 速度超标 = 真应急
- 才叫该叫人

关键判断

告警的本质不是"指标超了某个数"，是"错误预算正在以异常的速度被烧掉"。同样 P99=4s，高峰期偶尔一下，是正常波动（在预算内）；持续半小时狂烧预算，才是真应急。这个转变让告警从"任何一次抖动都叫"变成"只在真有问题时叫"——告警疲劳的根，就是没有错误预算这个概念。

THRESHOLD VS SLO

阈值告警 vs SLO 告警——逐维度对照

Google SRE / AWS / Anthropic 都用 SLO 这套

维度	阈值告警（错）	SLO 告警（对）
基础	某指标 > X 就告警	某 SLO 的错误预算消耗 > Y%
抗噪能力	弱（任何一次抖动都叫）	强（看累积消耗）
可量化	"P99 > 2s"	"99% < 3s，当前达成 99.2%"
与业务挂钩	没有	SLO 直接对应业务可接受度
告警密度	高（误报多）	低（只在真有问题时）
可解释	不可	"还剩 35% 错误预算"

老司机说

阈值告警最大的问题是"和业务脱钩"——P99 超 2s 到底要不要紧？没人说得清，全凭拍。SLO 告警永远能用一句业务语言解释清楚："我们承诺 99% 请求 3 秒内，现在这个月的错误预算还剩 35%，按当前速度 6 小时烧完"——这话 oncall 听得懂、老板也听得懂。

FIVE SLO CATEGORIES

Copilot 的 5 类核心 SLO + 错误预算——每类配明确数字和优先级

模糊的 SLO ("答得快一点") 不能驱动告警，必须是可测量的数字

SLO 类别	SLI	SLO 目标	错误预算	优先级
可用性	成功响应率	99.5%	0.5%/月 = 3.6h	P1
延迟	P99 端到端	< 3s	5%/月超时	P2
质量	Faithfulness (在线抽样)	> 0.85	15%/月低质	P2
合规	PII 泄露次数	= 0	零容忍	P0
成本	日均 cost/query (GPT-5 / Claude Sonnet 4.6)	< \$0.05	20%/月预算	P3

老司机说

注意合规这行——PII 泄露 SLO 是"= 0、零容忍、P0"，和其他可以"花预算"的 SLO 本质不同。其他 SLO 是"允许一定比例失败"的连续优化，合规是"一次都不行"的红线。别把它们用同一套阈值逻辑——可用性可以容忍 0.5% 失败，PII 泄露一次就得电话叫醒所有人。SLO 必须配明确数字，"差不多就行"驱动不了告警。

BURN RATE ALERT

错误预算燃烧速度告警——Google SRE 经典做法，快慢双烧

Fast burn 抓突发故障，Slow burn 抓慢慢退化

```
# observability/alerts/burn_rate.yaml
alerts:
- name: copilot_availability_burn_fast
  # 快烧：照这速度 < 6 小时烧光 30 天预算
  expr: |
    sum(rate(copilot_failed[1h])) / sum(rate(copilot_total[1h]))
    > 14.4 * 0.005 # 14.4x baseline burn rate
  for: 5m
  severity: P1
  routes: [pagerduty: oncall, slack: "#ai-alerts"]

- name: copilot_availability_burn_slow
  # 慢烧：照这速度 < 3 天烧光预算
  expr: |
    sum(rate(copilot_failed[6h])) / sum(rate(copilot_total[6h]))
    > 6 * 0.005 # 6x baseline burn rate
  for: 1h
  severity: P2
  routes: [slack: "#ai-alerts"]

- name: copilot_pii_leak # 零容忍红线
  expr: sum(rate(copilot_pii_leak[1m])) > 0
  for: 1m
  severity: P0
  routes: [pagerduty: security, phone: compliance-lead, slack: "#critical"]
```

老司机解读

14.4x baseline · Google SRE 经典公式：0.005 预算 × 14.4 倍速 = 6 小时烧光 30 天预算，这才叫真应急

Fast burn(5m) + Slow burn(1h) 双告警 · 既抓突发、又抓慢退化——这就是“多窗口多燃烧率”，2026 仍是 SRE 现行最佳实践

PII 零容忍 → P0 + 电话 · 红线一例就炸

for: 5m 防抖 · 持续 5 分钟才告警，避免单次抖动误报

不同优先级不同路由 · P0 电话 / P1 PagerDuty / P2 Slack——不让 oncall 被淹

ALERT SEVERITY

4 级告警分类 + 路由策略——"什么都重要 = 什么都不重要"

没有分级 = 真正事故被淹没在噪音里

级别	触发条件	响应 SLA	路由	示例
P0 红线	合规 / 资金 / 数据丢失	< 5 min 接电话	PagerDuty 电话 + SMS	PII 泄露 / 退款异常
P1 高	可用性 / 重大功能损坏	< 15 min	PagerDuty + Slack	错误率突增 / 主链路挂
P2 中	延迟 / 质量 / 部分功能	< 1h	Slack + Email	P99 退化 / 某工具失败
P3 低	趋势 / 成本 / 预警	< 8h (工作日)	Email + 日报	成本上涨 / 慢退化

踩坑提醒

Slack 群里"@channel"是最大的反模式——所有告警一个通道、一个响度，几天后全员屏蔽，等于没有告警。每一级必须有明确的"响应 SLA + 路由通道"：P0 必须电话打醒人，P3 进日报第二天看就行。分级的本质，是把团队有限的"被打扰额度"留给真正重要的事。

ON-CALL HANDOFF

告警 → Oncall → 处理的完整链路——告警必须自带上下文

oncall 收到告警不该再去查"为什么告警", 上下文要直接送到眼前

1. 告警触发, 自带完整上下文

```
alert: copilot_availability_burn_fast severity: P1
context:
  slo: availability > 99.5%
  current: 98.7% (burning at 14.4x)
  affected_traces: 2,847 (last 1h)
  top_error_types: [TOOL_TIMEOUT: 1502, LLM_429: 894]
  sample_trace_ids: [trace-abc, trace-def, trace-ghi] # ← 关键
```

2. 路由 → on-call (PagerDuty 电话 + Slack #ai-alerts)

3. Oncall 三步走

- # ① 看告警自带上下文 (上面)
- # ② 点 sample_trace_ids 跳 Phoenix 看具体 trace
- # ③ dashboard 查 Day-over-Day, 决定 rollback / hotfix

4. 处理 (Week 8 release manifest)

```
$ omni rag release rollback rag-v2026.05.17-003 \
  --reason "INC-2026-0518-001 (auto-paged: availability burn)"
```

5. incident 状态机自动追踪

triggered → ack → mitigated → resolved (全程时间戳 = MTTR)

老司机解读

告警自带 slo / current / sample_trace_ids · oncall 不用再去查"为什么告警"——上下文直接送到

sample_trace_ids 直接点进 Phoenix · 3 步从告警到 trace 详情, 这是上一讲钻取链的延续

rollback 带 --reason 关联告警 ID · 事后复盘能完整还原"为什么回滚"

incident 状态机 · 让"告警生命周期"也可观测, 能算 MTTR (平均修复时长)

这套就是 Google SRE 的 On-Call Excellence 标准实践

INDUSTRY 2026

SLO + 告警 = 现代 SRE 的工程基础——没 SLO 的告警是噪音机器

没有错误预算的团队，永远是救火队

来源	做法	启示
Google SRE	SLO + 错误预算 + 多窗口多燃烧率告警	系统稳定性的"科学基础"
PagerDuty / Datadog	告警→ack→升级→复盘全流程产品化	别自己造 incident 工作流
Anthropic / OpenAI	LLM API 给明确 SLO (99.9% / P99<5s)	可参考它做你的内部 SLO
Grafana SLO / Sloth / Pyrra	开源把 burn-rate 告警做成开箱即用	不用手写 Prometheus 表达式

记死

2026 年的共识 → 没有 SLO 的告警 = 噪音机器；没有错误预算的工程团队 = 救火队。SLO 是从"混乱救火"到"可控运营"的关键。一个延伸提醒：EU AI Act 对高风险 AI 要求"全生命周期自动记录日志、留存 ≥ 6 个月"——你这套 trace + 告警 + incident 记录，正好是合规审计要的东西。可观测不只是工程提效，2026 也是合规底座。

PITFALLS

告警设计的 5 个常见反模式——"没错误预算 + 没分级"最致命

团队疲劳之后，真正事故必被漏

反模式	具体表现	后果	正确做法
阈值拍脑袋	"P99 > 2s"	高频误报	从 SLO 反推 burn rate
没有分级	所有告警都 @channel	团队脱敏屏蔽	4 级 + 不同路由
没有错误预算	"出错就告警"	正常波动也叫	看预算消耗速度
不接 incident	告警不关联 trace	事后查不到	自带 sample_trace_ids
SLO 拍脑袋	"99.99% 可用"（业务不需要）	过度工程 + 浪费	从业务诉求出发

踩坑提醒

"SLO 拍脑袋定 99.99%"是个反直觉的坑——很多人觉得 SLO 越高越好，其实过高的 SLO 是巨大的浪费。99.9% 到 99.99% 的成本可能是 10 倍，而你的客服 Copilot 客户根本感知不到那 0.09%。SLO 要从业务"能接受多差"反推，不是从"我想多好"出发。过度可用性，是把钱烧在客户根本不在乎的地方。

LESSON 05 · BAD CASE POSTMORTEM

Bad Case 复盘标准化：让一次事故成为团队的"工程财富"，不是"再来一遍"

前4节让故障被发现，这一讲让故障被定位、被沉淀

真实场景 · 永远在救火的团队

客户报："昨天那条回答指错了引用。" 90% 的团队是怎么处理的？工程师手动翻日志 → 凭经验推测 → 改个 prompt → 上线 → 客户继续抱怨 → 再翻日志.....永远在救火，永远在重复同一类错。生产级团队不一样：他们把每个 bad case 当"工程学习机会"——用 trace 证据链定位根因 → 加进反例库 → 写进 Runbook → 修完跑回归验证。同样是处理 bad case，一个是"扫掉它"，一个是"从它身上赚一笔复利"。这一讲讲清楚：怎么把 bad case 复盘，做成团队级的工程财富积累。

咱们这节聊 4 件事：

- 为什么 bad case 复盘必须是"标准流程"——不能靠工程师个人习惯

- Bad Case 复盘 5 步：Detect / Triage / Locate / Fix / Verify，每步有 SLA + 产物

- 基于 trace 的"根因层定位"——5 分钟决策树，新人也能定位

- 把 bad case 自动加进 Week 11 反例库 + Week 9 Runbook——形成复利

BAD CASE = GOLD

Bad Case 不是"麻烦事"——是 AI 系统持续改进的"复利燃料"

新手想着扫除它，生产级团队把它当金矿

进反例库

- 加进 Week 11
- adversarial 样本
- 下次回归自动拦
- ——生产 bad case 是最好的反例

更新 Runbook

- 触发 Week 9
- Runbook 更新
- 下次同类 5 分钟解决
- ——经验沉淀成工艺

进微调集

- 可能进入
- 模型微调数据
- 长期改基础能力
- ——免费的金标准数据

关键判断

Bad case 是免费的"金标准数据"——客户帮你标好了"这个答案是错的、正确应该是什么"，这是花钱都买不到的。只有"会用"的团队才能享受这份复利：一个 bad case 进了反例库，它就永远在替你站岗，下次任何改动想再犯同类错，CI 直接拦住。把 bad case 当垃圾扫掉的团队，永远在为同一个坑反复付费。

FIVE-STEP POSTMORTEM

Bad Case 复盘的 5 步标准流程——每步有明确 SLA + 产物

没产物 = 这次复盘没真的发生

步骤	具体动作	SLA	关键产物
1. Detect	收集 bad case + trace_id + 客户反馈	< 1h	incident ticket
2. Triage	严重度评估 + 路由到 oncall / 工程师	< 30min	严重度标签
3. Locate	基于 trace + span 定位根因层	< 2h	根因分析报告
4. Fix	改 prompt / 索引 / 工具 / Skill	< 24h（紧急）	PR + 回归测试
5. Verify	加进反例库 + 跑回归 + 更新 Runbook	< 48h	postmortem doc

老司机说

为什么要标准化、不能靠工程师习惯？因为“靠习惯”意味着资深的人在时能定位、人一走就抓瞎，而且每次复盘深浅不一。标准流程 + 明确产物，让“复盘”从一项个人技能变成团队能力。最容易被省掉的是第 5 步 Verify——很多团队修完就完了，不加反例、不跑回归、不更新 Runbook，下次同类 bad case 必然重发。没有第 5 步，前 4 步白做。

ROOT CAUSE LOCATOR

Trace 根因定位决策树——5 分钟法则，新人也能定位

没 trace 靠"猜测 + 复现 + 改一个看一次"，半天到一天；有 trace，按树走 5 分钟



关键判断

这条决策链最大的价值，是让排障"不再依赖资深工程师"。过去 bad case 来了，只有那个做了三年的老人能定位；现在有了 trace + 决策树，入职两周的新人也能在 5 分钟内说出"根因在生成层、evidence_count=0、是 LLM 编造、对应 Week 8 加严 prompt"。把"靠经验猜"升级成"按 trace 查"——平均排障时间能砍掉 80%（小时级→分钟级），这是团队可观测能力从"个人英雄"到"系统化"的关键一跃。

ROOT CAUSE MAP

Bad Case 5 类常见根因 + 对应修复——这张表贴墙上、钉 Slack 置顶

oncall 和工程师复盘的"诊断速查表"

根因层	症状	Trace 看什么	修复（哪周学过）
检索漏召	答案缺关键事实	retrieve.* hits 很少	Week 8 hybrid + Week 7 chunk
重排丢失	召回了但排到很后	rerank.kept 异常	Week 8 调阈值 / 换 reranker
LLM 编造	答案有但引用错	evidence_count = 0 / mismatch	Week 8 加严 prompt + Schema
工具失败	Agent 报错 / 部分响应	tool.* status = ERROR	Week 10 工具契约 / fallback
HITL 阻塞	响应卡死	hitl.wait 时长异常	Week 10 异步 + SLA

老司机说

这张表把整门课串起来了——可观测（Week 12）负责"看见根因在哪层"，前面几周负责"怎么修那一层"。trace 看 evidence_count=0 → 生成层 LLM 编造 → 回到 Week 8 加严 prompt；trace 看 retrieve hits 少 → 检索漏召 → 回到 Week 7/8 调 chunk 和 hybrid。可观测不是孤立的一周，是把前 11 周的能力"接上诊断回路"。

POSTMORTEM TEMPLATE

OmniSupport 的标准复盘模板——7 段，每段都有明确产物

不是"开会聊聊",是"完成的事项清单"

```
# postmortems/INC-2026-0518-001.md
## 1. Summary
- Incident: 客户报"退款流程"回答错引用 | Severity: P2
- Affected: 5 客户 | Duration: 检测→修复 = 18h
## 2. Trace Evidence
- trace_id: trace-20260518-001 (Phoenix link: ...)
- rag.retrieve.hybrid: OK hit=23
- rag.rerank.cross: OK kept=5
- llm.generate: WARN evidence_count=0 ← 根因
## 3. Root Cause
- 层级: 生成层 (LLM 编造)
- 具体: Prompt v3.2.0 缺"必须引用 evidences"硬约束
## 4. Fix
- PR #142: prompt → v3.2.1, 加 evidences 约束
- 评测回归: faithfulness 0.85 → 0.91
- 反例库: 本 case 加入 evals v2.3.0 → v2.3.1
## 5. Verify
- [x] 合并 + Canary 5% 30min SLO 正常
- [x] Runbook(Week 9) 已更新
## 6. Lessons / 7. Action Items
- Prompt 改动必须先跑 adversarial 反例库
- 加 evidence_count=0 实时告警 (已进 alerts.yaml)
```

老司机解读

7 段每一段都有明确产物 · 不是"开会聊聊", 是可勾选的事项清单

Trace Evidence 直接附 Phoenix 链接 · review 时一键看现场, evidence_count=0 一眼定位根因

Fix 段关联 PR + 回归结果 · "改了没、有效没"可验证 (faithfulness 0.85→0.91)

Lessons Learned 是金子 · 这些教训直接变成 Week 9 Runbook 的更新

Action Items 必须 100% 完成才能 archive · 否则下次同类事故必然重发

BAD CASE → REGRESSION

把 Bad Case 自动加进 Week 11 反例库——让每个 case 产生复利

生产 bad case 就是 Week 11 评测集 "Adversarial 样本" 最好的来源

```
# tools/badcase_to_eval.py
def add_badcase_to_evalset(incident_id, trace_id, user_query,
                           expected_answer, actual_bad_answer, root_cause):
    sample = {
        "sample_id": f"adv-{incident_id}",
        "category": "adversarial",
        "subcategory": root_cause,      # "llm_hallucinate" 等
        "query": user_query,
        "expected_answer": expected_answer,
        "expected_evidences": pull_from_trace(trace_id),
        "actual_bad_answer": actual_bad_answer, # 记录"曾经的错"
        "source": {"type": "production_incident",
                   "incident_id": incident_id, "trace_id": trace_id,
                   "trace_url": f"https://phoenix/trace/{trace_id}",
                   "postmortem_url": f"./postmortems/{incident_id}.md"},
    }
    cur = json.load(open("evals/v2.3.0.json"))
    cur["samples"].append(sample)
    cur["version"] = bump_patch(cur["version"]) # 2.3.0 → 2.3.1
    # 加完立刻跑回归，验证"修复有效"
    result = run_eval(cur, trigger_pr=False)
    assert result["samples_by_id"][sample["sample_id"]]["pass"]
    return cur
```

老司机解读

actual_bad_answer 记录"曾经的错" · 让团队知道这条为什么是反例

source 段串起 incident / trace / postmortem · 一键跳回所有上下文

bump_patch 自动版本号 · eval set 像代码一样版本化（Week 11 学过）

assert result["pass"] · 加完立刻跑——"修复有效"是加入反例库的前置条件

这一刻起，每个客户报的 bad case 都自动沉淀成团队的工程财富——这才叫复利

INDUSTRY 2026

Bad Case 复盘文化 + 反例库复利 = AI 系统持续改进的核心引擎

这比任何"模型微调"都更重要

来源	做法	启示
Google SRE	Blameless Postmortem（不归罪个人，只追根因）	让团队敢上报 bad case，不藏
Anthropic	把 adversarial 反例库做成系统核心	每次模型升级对照反例库回归
OpenAI	内部 adversarial bench，上线必跑红队反例	合规 + 质量双门禁
Latitude / Braintrust（2026）	trace → 失败标注 → 自动生成 eval → 回归	闭环：生产故障自动变评测

记死

2026 年的共识 → bad case 复盘 + 反例库复利，是 AI 系统持续改进的核心引擎，比追新模型更重要。一个值得跟的 2026 趋势：Latitude、Braintrust 这类平台正在把"生产 trace → 失败标注 → 自动生成 eval → 回归"做成闭环——生产里的每一次故障，自动变成评测集里的一条新反例。这就是 Week 11（评测）+ Week 12（可观测）合体的终极形态。

WEEK 12 MAP

Week 12 五节完整能力栈——从"黑盒"到"x 光"

故障定位从小时级降到分钟级



关键判断

这 5 节连起来，就是一台完整的"故障定位显微镜"：有协议（OTel/OpenInference）、有细节（Spans）、有总览（Dash）、有主动报警（Alert）、有事后沉淀（Replay）。Copilot 从"出了事两眼一抹黑"，变成"5 分钟定位 + bad case 自动复利"。前 12 周让它"答得稳 + 办得对 + 看得见"。Week 13 起升级 RAG——引入图结构，让它能回答"跨文档归纳 / 多跳推理"这类复杂问题。

WEEK 12 · THE END

你已经给 Copilot 装上了"X 光"

从 OTel 协议到 5 个仪表盘，从 SLO 告警到 Bad case 复盘——Copilot 第一次具备了"故障 5 分钟定位 + Bad case 自动复利"的可持续运营能力

本周交付物（已 push 至 GitHub 仓库）

OTel Setup

pipelines/observability/setup.py

Traced RAG

services/rag/traced.py

Dashboards

observability/dashboards/*.py

Burn Rate Alerts

observability/alerts/burn_rate.yaml

Postmortem Tpl

postmortems/template.md

Bad-Case → Eval

tools/badcase_to_eval.py

下周预告

下周 → Week 13 · GraphRAG：让 Copilot 能回答"跨文档归纳 / 多跳推理"这类复杂问题。评测让你知道好不好，可观测让你知道出事了在哪，而 GraphRAG，让它能答以前答不了的难题。